COS30018 – Intelligent Systems

Task C.3 – Data processing 2

First I “pip install mplfinace“ and then imported it as mpf because I will use it to plot the candlestick chart.

```
def plot_candlestick(df, n=1, title="Candlestick Chart"):
#-----------------------------------------------------------------------------
# Resample data if n > 1 (group n trading days into 1 candle)
#-----------------------------------------------------------------------------
    if n > 1:
        df_resampled = df.resample(f"{n}B").agg({
            "Open"  : "first",
            "High"  : "max",
            "Low"   : "min",
            "Close" : "last",
            "Volume": "sum"
        }).dropna()
    else:
        df_resampled = df.copy()

#-----------------------------------------------------------------------------
# Plot the candlestick chart
#-----------------------------------------------------------------------------
    mpf.plot(
        df_resampled,
        type            = "candle",
        style           = "charles",
        title           = title,
        ylabel          = "Price (AUD)",
        volume          = True,
        mav             = (20, 50),
        show_nontrading = False,
    )
```

This is what I have implemented for the candlestick chart. It will take these parameters:

- df: dataframe with DataTimeIndex and columns like “Open”, “Close”, “High”, “Low”, “Volume” with the format outputted by load_and_process_data function.
- n: the numbers of days each candle represents, n = 1 meaning that each candle represents the data of 1 day, similarly to n = 2,3,4…
- title: the title of the chart.

Resample Part:

resample(f"{n}B") will group data into n workdays, skip the Saturday and Sunday (B = Business Day). After that, the “agg” will calculate the values of the groups using the financial rules:

- High: the max value in the n days period.
- Low: the min value in the n days period.
- Open: the starting value of the data in the n days period.
- Close: the last value of the data in the n days period.

Plotting Part:

The “mpf.plot” take these parameters that need to be explained:

- type: the type of what shape it will draw, it can be change to “ohlc” and “line” if wanted.
- style: the color style of the candlesticks that represent the data flow, “charles” here means the classic green/red color style of the candlesticks.
- volume: should the volume be drawn at the bottom of the window.
- mav: draw 2 additional moving average lines for 20 days and 50 days so that we can watch the flow of the data in the short and medium periods.
- show_nontrading = false: hide days that the marketplace stop working (the weekends and holidays).

```
def plot_boxplot(df, n=20, title="Boxplot Chart"):
#-----------------------------------------------------------------------------
# Slice Close prices into non-overlapping windows of size n
#-----------------------------------------------------------------------------
    close_prices = df["Close"].values

    windows, labels = [], []
    for i in range(0, len(close_prices) - n + 1, n):
        windows.append(close_prices[i:i+n])
        labels.append(df.index[i].strftime("%Y-%m-%d"))

#-----------------------------------------------------------------------------
# Plot the boxplot
#-----------------------------------------------------------------------------
    plt.figure(figsize=(14, 6))
    plt.boxplot(
        windows,
        labels       = labels,
        patch_artist = True,
        boxprops     = dict(facecolor="lightblue", color="navy"),
        medianprops  = dict(color="red", linewidth=2),
        whiskerprops = dict(color="navy"),
        capprops     = dict(color="navy"),
        flierprops   = dict(marker="o", color="gray", markersize=3),
    )
    plt.xticks(rotation=45, ha="right")
    plt.title(title)
    plt.xlabel(f"Window start date (every {n} days)")
    plt.ylabel("Close Price (AUD)")
    plt.tight_layout()
    plt.show()
```

This is what I have implemented for the boxplot chart. It will take these parameters:

- df: dataframe with DataTimeIndex and columns like “Open”, “Close”, “High”, “Low”, “Volume” with the format outputted by load_and_process_data function.
- n: the numbers of days each box represents, n = 1 meaning that each box represents the data of 1 day, similarly to n = 2,3,4…
- title: the title of the chart.

Windows Creating Part:

The windows will jump n steps each time, result in a non-overlap state. For example, if n=20 then the first box will contain the data from day 0 to day 19, then the next box will be from day 20 to 39, and so on. The label of the boxes will be the start day of each window, with the format of "%Y-%m-%d" so that it is easier to read when there are a line of boxes.

Plotting Part:

The “plt.boxplot” take these parameters that need to be explained:

- patch_artist = True: color the inside of the boxes, if it is “False” then there will only be the outlines of the boxes.
- boxprops = dict(facecolor="lightblue", color="navy"): because I set that the boxes should have color inside, so “lightblue” will be the color of the inside of the boxes, and “navy” will be the color of the outlines.
- medianprops = dict(color="red", linewidth=2): the median line will have the color “red” with the width of the line be 2 pixels. The “median” is a line inside the box that represents the median value (the middle value) of the data.
- whiskerprops = dict(color="navy"): the whiskers (top and bottom) of the boxes will have the color “navy”. The height of the box (IQR) is calculated by using Q3 – Q1, with Q3 is the middle value from the median to the largest value and represents the upper limit of the box, and the Q1 is the middle value from the median to the smallest value and represents the lower limit of the box. The whiskers are calculated by multiply the IQR with 1.5.
- The outlier is the value that is outside of the whiskers. The outliers of the boxes are set to be drawn with the “circle” shape, have the color “gray”, 3 pixels big.

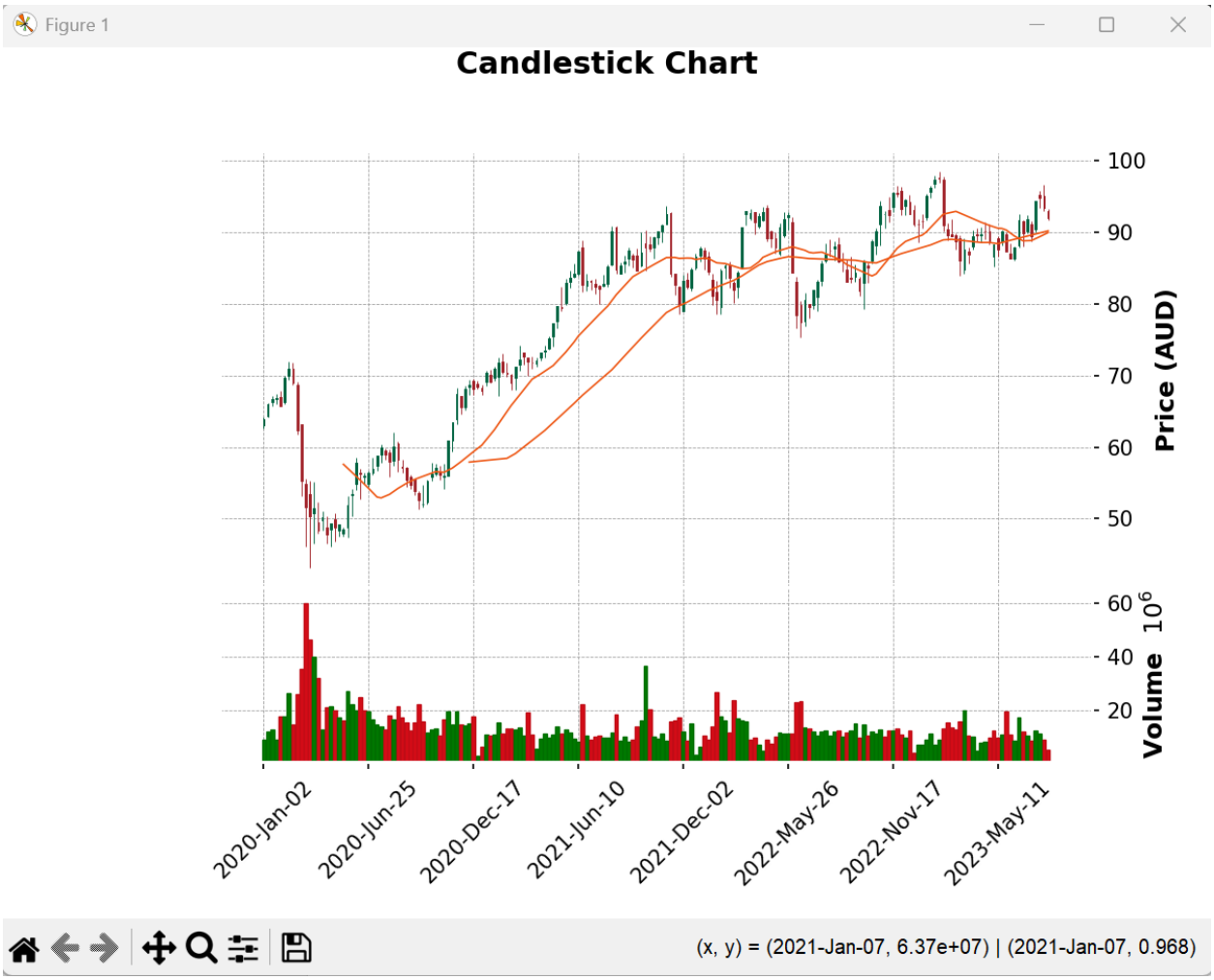
```
#-----------------------------------------------------------------------------
# Main
#-----------------------------------------------------------------------------
COMPANY    = "CBA.AX"
START_DATE = "2020-01-01"
END_DATE   = "2024-07-02"
PREDICTION_DAYS = 60

result = load_and_process_data(
    company         = COMPANY,
    start_date      = START_DATE,
    end_date        = END_DATE,
    nan_strategy    = "fill_ff",
    split_method    = "ratio",
    train_ratio     = 0.8,
    scale_features  = True,
    save_local      = True,
    prediction_days = PREDICTION_DAYS,
)

# Plot additional charts
plot_candlestick(result["train_df"], n=5, title="Candlestick Chart")

plot_boxplot(result["train_df"], n=20, title="Boxplot Chart")
```

I made the program draw those 2 charts right after the “load_and_process_data” function so that we can have a better understanding of the data before using it to train the model.



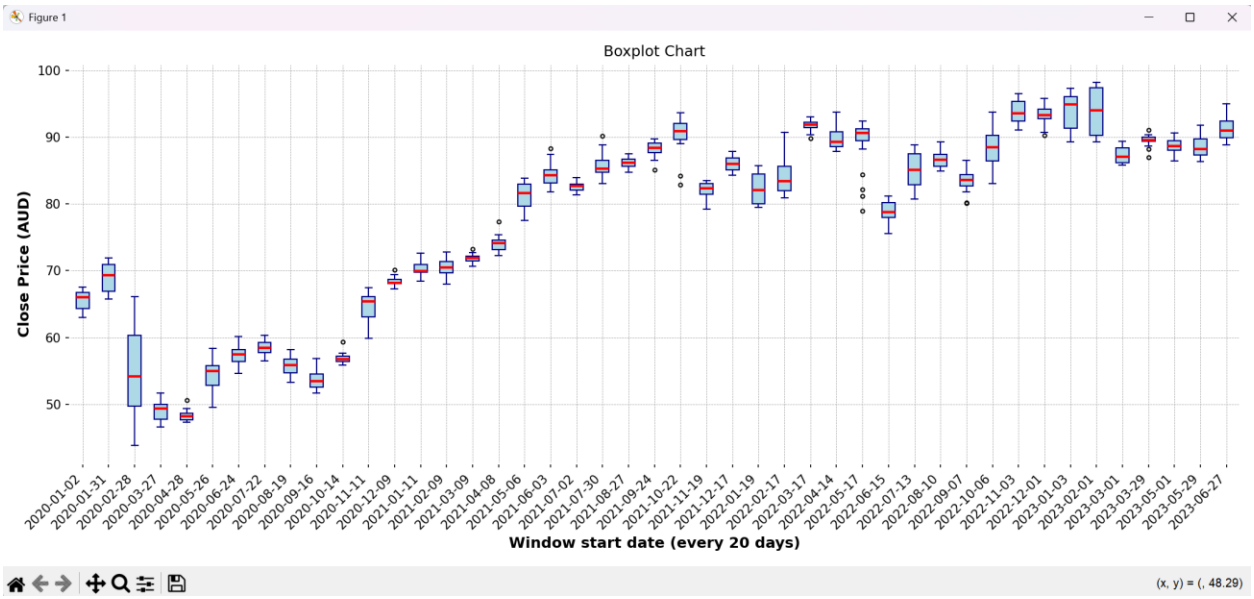


Figure 1
Boxplot Chart
Close Price (AUD)
100
90
80
70
60
50
2020-01-02
2020-01-31
2020-02-28
2020-03-27
2020-04-28
2020-05-26
2020-06-24
2020-07-22
2020-08-19
2020-09-16
2020-10-14
2020-11-11
2020-12-09
2021-01-11
2021-02-09
2021-03-09
2021-04-08
2021-05-06
2021-06-03
2021-07-02
2021-07-30
2021-08-27
2021-09-24
2021-10-22
2021-11-19
2021-12-17
2022-01-19
2022-02-17
2022-03-17
2022-04-14
2022-05-17
2022-06-15
2022-07-13
2022-08-10
2022-09-07
2022-10-06
2022-11-03
2022-12-01
2023-01-03
2023-02-01
2023-03-01
2023-03-29
2023-05-01
2023-05-29
2023-06-27
Window start date (every 20 days)
(x, y) = (, 48.29)